2026 第九届传智杯全国大学生数智创新与 AI 应用大赛

AI WEB 网页开发挑战赛 · 团队赛

极构·需求诊断顾问

技术文档

把「我想做个系统」，变成一份能直接拿去开发、询价、立项的需求报告。

沧州交通学院 · 极构团队 · B 组

成员：贾宇航、吴思彤、马翌焜

指导教师：王玉潇

2026 年 11 月

文档信息

文档名称	极构 • 需求诊断顾问 技术文档
文档版本	v2.0
编制日期	2026 年 11 月
编写团队	沧州交通学院 • 极构团队
适用读者	评委、指导教师、研发与测试人员
代码仓库	https://github.com/ynamlu/JiGou
配套材料	演示视频、静态页总览、技术文档

文档说明

本文记录极构的设计取舍、系统结构、AI 方案、关键实现与验证结果。写法尽量贴近工程现场：说清楚为什么这样做、哪里做了妥协、哪些结论有实测依据。

章节结构

- 项目背景与目标
- 需求与范围
- 系统结构
- 模型接入与 Agent
- 检索增强
- 核心功能
- 安全与异常
- 工程化与部署
- 测试与质量
- 局限与计划

目　录

CONTENTS

一、项目背景与目标 4
1.1 一个反复出现的场景 4
1.2 我们要解决的问题 4
1.3 产品边界 4
1.4 设计原则 5
二、需求与范围 6
2.1 用户任务 6
2.2 功能范围 6
2.3 本期不做的事 7
2.4 非功能要求 7
三、系统结构 8
3.1 运行形态 8
3.2 前端组织 8
3.3 后端组织 8
3.4 数据放在哪里 9
3.5 流式协议 9
四、模型接入与 Agent 11
4.1 为什么走 OpenAI 兼容协议 11
4.2 Agent 的职责 11
4.3 会话与问卷状态 11
4.4 结构化输出的容错 12
4.5 提示词的分工 12
五、检索增强 13
5.1 知识库的来源 13
5.2 切片与索引 13
5.3 两条检索路径 13
5.4 降级不是口号 14
5.5 检索的局限 14
六、核心功能实现 15
6.1 账号与鉴权 15
6.2 问卷 15
6.3 自由对话 15
6.4 报告与待确认事项 16
6.5 评分、交付物与 A/B 方案 16
6.6 验收与版本 16
6.7 导出、模板库与适配 17
七、安全与异常 18
7.1 输入审查 18
7.2 提示词边界 18
7.3 输出脱敏 18
7.4 故障与降级 18
7.5 隐私边界 19
八、工程化与部署 20
8.1 一条命令启动 20
8.2 环境变量 20
8.3 依赖与版本 21
8.4 构建与静态托管 21
九、测试与质量 22
9.1 自动化检查 22
9.2 接口回归 22
9.3 真实模型链路 22
9.4 浏览器检查 23
9.5 已知问题 23
十、局限与后续 24
10.1 当前局限 24
10.2 下一步 24
附录 A　目录结构 25
附录 B　接口清单 26
附录 C　团队信息 27

页码与正文实际分页一致；如需修改内容，请同步更新目录页码。

一、项目背景与目标

1.1 一个反复出现的场景

我们最初想做这个工具，是因为身边总能遇到类似的对话。有人准备在学校旁边开一家奶茶店，想做个扫码点单的小程序；有人手里有一个客户名单，想做个简单的进销存；也有人只是说「我想做个平台，能不能帮我看看要多少钱」。再往下问，问题就开始卡住：要不要会员体系，支付资质有没有，订单状态怎么流转，预算三万和十五万差在哪里。

这些问题的答案并不神秘，难的是没有人替需求方把它们逐个问出来。外包公司会问，但提问本身有成本，而且双方立场不同。通用聊天机器人也能聊，但回答往往停在「建议你明确目标用户」这种层面，落不到具体的功能、预算和验收标准上。

我们想做的，是一个站在需求方这一侧的提问者。它不教人写代码，也不替人做决定，只负责把模糊的想法拆成可以讨论、可以询价、可以验收的材料。

1.2 我们要解决的问题

项目要解决的核心问题可以拆成三层。

第一层是信息不完整。用户往往只描述了「做什么」，没有说明「给谁用」「必须有什么」「什么时候上线」「能花多少钱」。系统需要主动追问，而不是等用户把所有信息一次性写全。

第二层是判断缺依据。需求是否合理、报价是否离谱、还缺哪些资质，这些判断依赖经验。我们把常见的软件开发知识整理成 12 篇文档，让模型在回答前先检索，尽量做到有出处、有边界。

第三层是结果不可用。聊完之后如果只留下一段聊天记录，用户还是不知道下一步做什么。系统最终要产出报告、待确认清单、评分、PRD、询价清单、成本工期和验收凭证，让诊断结果能够进入真实的沟通流程。

1.3 产品边界

极构不是代码生成器，也不是项目管理工具。它不写业务代码，不替代律师或财务，不承诺报价就是最终成交价。它做的事情只有一件：在开发开始之前，帮助需求方把问题想清楚，并把想清楚的部分固定下来。

这个边界是刻意划的。比赛题目强调 AI 是产品的核心，我们理解这句话的意思不是「每个页面都塞一个聊天框」，而是产品的主要价值必须由模型能力产生。极构如果去掉 AI，剩下的只是一个空表单；保留 AI，它才是一个会追问、会检索、会归纳的诊断流程。

1.4 设计原则

整个项目围绕三条原则做取舍。

一是先保证能跑，再追求复杂。比赛现场的网络、模型网关和评审机器都不完全可控，所以系统采用本地单进程、单文件数据库、可选向量库的结构。没有外部向量服务时，检索会自动落到本地的 BM25，功能不中断。

二是把不确定性收在边界内。模型输出不稳定，我们就在解析层做容错：去代码围栏、提取 JSON、逐字段校验、失败时返回空结构或默认问题。模型网关不可用时，页面仍然能打开，接口给出明确的提示，而不是白屏。

三是让用户看见系统在做什么。对话不是等一大段文字突然出现，而是逐字返回；检索和工具调用会显示状态；生成可以中止；报告可以修订后重新验收。这些细节不改变模型能力，但决定了产品是否像一个可以使用的工具。

二、需求与范围

2.1 用户任务

我们把目标用户的任务归纳为四类，后面的功能设计都围绕这四类任务展开。

用户任务	用户心里的问题	系统提供的能力
把想法说清楚	我到底要做什么，哪些是必须的	逐题问卷、自由对话、结构化追问
判断还缺什么	我是不是漏了支付、备案、会员体系	对照知识库做完整性诊断，列出缺失项
拿到可用的材料	能不能给我一份能发给开发方的东西	诊断报告、PRD、询价清单、成本工期、A/B 方案
留下书面凭据	后面改需求会不会扯皮	待确认事项逐条拍板、验收版本、指纹与 diff

2.2 功能范围

功能按「账号、诊断、报告、评估、交付、验收、内容、导出」八块组织。优先级用 P0/P1/P2 表示，P0 是完整走通一次诊断所必需的，P1 影响交付质量，P2 是体验补充。

模块	功能	说明	优先级
账号	注册、登录、改密、注销	手机号加强密码，Bearer Token 鉴权，按账号隔离数据	P0
诊断	逐题问卷	根据已答内容生成下一题，支持单选、多选、文本和补充说明	P0
诊断	自由对话	NDJSON 流式输出，展示检索与思考状态，可停止生成	P0
报告	需求诊断报告	项目概述、已具备、还缺失、待确认、建议下一步	P0
报告	待确认事项	抽取模糊、缺失、冲突项，逐条确认并记录决策	P0
评估	六维成熟度评分	形态、用户、功能、预算、时间、合规，附风险提示	P1
交付	PRD、RFP、成本工期	基于报告生成三份可直接使用的材料	P1
决策	A/B 方案	MVP 与完整版对比，给出推荐与理由	P1

模块	功能	说明	优先级
验收	版本与凭证	SHA-256 指纹、版本号、行级 diff、凭证导出	P1
内容	模板库与案例库	6 套模板、2 个案例，支持搜索、预览、导出和带入诊断	P2
内容	我的图纸	按账号归档，支持搜索、删除和状态徽章	P1
导出	Markdown、PDF、长截图	服务端 PDF 排版，前端生成高清长截图	P2

2.3 本期不做的事

明确不做什么，和明确做什么同样重要。本期没有做多租户和团队协作，一个账号的数据只属于一个浏览器和一个本地数据库；没有做云端同步，换设备后历史报告为空是预期行为；没有接入语音和图片输入，诊断只处理文本；没有做自动报价下单，成本工期只是参考区间；也没有把模型输出直接当成法律或财务意见。

这些限制有的来自比赛周期，有的来自本地单机的产品形态。我们在文档里保留它们，是为了让后续维护者知道哪些是能力边界，哪些只是还没做。

2.4 非功能要求

维度	要求	实现方式
性能	首屏可交互，对话逐字出现	前端单文件构建、路由懒加载，后端 NDJSON 流式返回
可用性	外部模型或向量服务异常时仍能演示	检索自动降级 BM25，模型异常返回可读提示
安全	密码不落明文，接口需鉴权，Key 不进仓库	PBKDF2-SHA256 加随机盐，Bearer Token，环境变量与忽略文件
兼容	PC 和手机浏览器都能用	内容驱动的响应式布局，窄屏改为单列
隐私	个人数据尽量留在本机	SQLite 单文件加浏览器本地存储，不依赖第三方云服务
可维护	一条命令启动，配置外置	run.py 负责虚拟环境、依赖、构建、端口和浏览器

三、系统结构

3.1 运行形态

系统是「浏览器单页应用 + 本地 FastAPI 服务 + 模型网关」的三层结构。浏览器负责交互、Markdown 渲染和本地存储；FastAPI 负责鉴权、AI 编排、检索、结构化输出和文件导出；模型网关只负责推理。生产模式下 FastAPI 直接托管前端构建产物，所以整个应用只需要一个 Python 进程，不需要 Nginx、Docker 或数据库服务。

这种形态适合比赛和单机交付，但也有明显代价：SQLite 不适合多实例并发，浏览器本地存储不跨设备，服务端的内存会话在重启后丢失。我们接受这些代价，换取的是评审现场一条命令就能启动。

3.2 前端组织

前端按「页面、组件、状态、接口」四层组织。页面在 `frontend/src/views`，包括登录、注册、对话、报告、模板库、历史和个人中心；组件在 `frontend/src/components`，把对话气泡、问题卡、评分卡、交付物卡、方案卡、确认弹窗等拆开；Pinia 管理账号、对话、问卷、报告归档和分析草稿；`src/api` 统一附加 Token、解析 NDJSON、处理 401。

路由使用 hash 模式，避免本地静态托管时的刷新 404。未登录访问受保护页面会被守卫拦到登录页，并在 URL 上带上 `redirect` 参数，登录后回到原页面。Markdown 统一由 `utils/markdown.js` 渲染，代码高亮、复制按钮和外链安全只实现一次。

响应式没有按设备类型判断，而是按内容宽度调整。桌面端是双栏 Hero 加顶部导航，移动端压成单栏并保留底部 Tab；表格和卡片在窄屏改为单列。这样在平板、分屏和浏览器缩放时也不会出现布局断裂。

3.3 后端组织

后端代码不多，职责划分比较直接。

模块	文件	职责
HTTP 与鉴权	`app.py`	路由、Token 校验、请求校验、流式响应、文件导出
AI 核心	`agent_core.py`	模型客户端、提示词、Agent、结构化生成、事件协议
检索	`retrieval.py`	BM25 索引、切片、停用词、标题加

模块	文件	职责
		权、同源去重
安全	`safety.py`	输入审查、注入识别、敏感信息脱敏
数据	`db.py`	账号、登录凭证、验收记录的 SQLite 读写
导出	`pdf_export.py`	Markdown 报告转 PDF
启动	`run.py`	虚拟环境、依赖、前端构建、端口选择、服务启动

`app.py` 不直接拼提示词，`agent_core.py` 也不处理 HTTP。这条边界让提示词调整和接口调整可以分开做，测试时也更容易定位问题。

3.4 数据放在哪里

数据分两处存放，这是本项目最需要解释清楚的设计之一。

数据	位置	原因
账号、密码哈希、登录 Token	`backend/data/jigou.db` 的 users、tokens 表	需要跨会话保留，且不能只存在浏览器
验收记录、版本、diff	`backend/data/jigou.db` 的 acceptances 表	作为书面凭证，换浏览器也应能核对
报告正文、评分、交付物、草稿	浏览器 `localStorage`	内容较大，且属于用户隐私，尽量不离开本机
知识库	`backend/kb/*.md`	纯文本，便于审阅和替换
向量库	`backend/vector_store/`（可选）	配置 embedding 后生成，不属于启动必需项

浏览器里的键名都带账号：`jg_session` 保存当前账号和 Token，`jg_archive:<手机号>` 保存报告归档，`jg_analysis_draft:<账号>` 保存分析草稿。同一浏览器切换账号时，各自只看得到自己的数据。SQLite 只保存需要跨会话和跨浏览器保留的三类数据，报告正文不进去。

3.5 流式协议

对话接口使用 NDJSON，一行一个 JSON 事件。前端不需要理解模型 SDK 的返回结构，只需要按 `type` 分发。

事件	含义	前端处理
`token`	正文增量	追加到当前气泡

事件	含义	前端处理
`thinking`	模型思考状态	显示思考提示，不写入正文
`tool_start`	开始调用知识库	显示「正在检索知识库」
`tool_end`	检索结束	收起检索状态，可展示命中来源
`done`	本轮结束	结束 loading，恢复输入
`error`	可读错误	以助手消息展示，不弹底层异常

选择 NDJSON 而不是 WebSocket，是因为这里的通信是单向的、一次请求对应一次生成，普通 POST 加流式响应已经够用。FastAPI 的 StreamingResponse 直接支持，前端用 fetch 的 reader 逐行解析，部署时也不需要额外的长连接网关。

四、模型接入与 Agent

4.1 为什么走 OpenAI 兼容协议

模型通过 OpenAI 兼容接口接入，默认网关是 `https://maas.qianwenaiapi.com/compatible-mode/v1`，默认型号 `deepseek-v4.1-flash-1017`。我们没有绑定某一家厂商的 SDK，原因很实际：比赛网关、云端 API 和本地模型之间需要能切换；LangChain 对兼容协议的支持最完整；请求和响应格式稳定，流式解析和错误处理可以复用。

所有模型参数集中在 `config.py`。Key 支持进程环境变量、`backend/.env` 和 `backend/local_config.py` 三种来源，读取优先级从高到低。`temperature` 默认 0，结构化输出时保持 0；`enable_thinking` 采用「没设置就不发送」的策略，避免不同网关对未知字段的兼容性差异。这个细节看起来小，但在换网关时省了很多排查时间。

4.2 Agent 的职责

自由对话由 LangChain 的 `create_agent` 构建，挂载一个 `search_knowledge` 工具和 `InMemorySaver` 会话记忆。Agent 的职责不是陪聊，而是按系统提示词完成三件事：判断信息是否足够、必要时检索知识库、按「已具备、还缺失、待确认」的结构输出。

工具调用参数和结果会通过事件协议实时推给前端。用户看到的不是一句「正在思考」，而是「正在检索知识库」，检索结束后再看到回答。这个反馈不改变结果质量，但能减少等待时的焦虑。

问卷、评分、交付物、A/B 方案这些一次性任务不走 Agent，而是走同步的结构化生成。原因是它们不需要多轮工具调用，走 Agent 反而会把工具调用和会话记忆带进本该稳定的 JSON 输出。我们把「需要多轮判断」和「需要一次成文」分成两条路径，前者用 Agent，后者用固定提示词加解析。

4.3 会话与问卷状态

会话记忆由 LangGraph 的 `InMemorySaver` 保存，按 `session_id` 区分。问卷答案另存在 `SESSION_QA` 字典里，生成报告时作为附加上下文拼进提示词。两者都不落库，服务重启后清空，这是本地单机场景下的有意选择：报告本身已经保存在浏览器，服务端不需要再维护一份对话历史。

这也带来一个限制：如果用户在生成报告前重启服务，问卷上下文会丢失。前端在这种情况下会提示重新开始，而不是假装还能继续。我们认为这比悄悄生成一份缺少上下文的报告更诚实。

4.4 结构化输出的容错

模型返回 JSON 时常见的三种意外是：包在代码围栏里、前后带解释文字、字段类型不对。我们的处理顺序是先去围栏，再用正则提取最外层 JSON，然后逐字段校验，非法项丢弃或回落默认值。

问卷出题失败时回落到一组默认问题，保证流程不中断；待确认项解析失败时返回空数组，前端展示空状态；评分数值裁剪到 0 到 100，非法等级回落到 medium；交付物和 A/B 方案解析失败时返回空结构。这里的原则是「宁可少显示，也不要把解析异常抛给用户」。

开发过程中遇到过一个典型问题：报告正文里包含花括号时，用 `str.format` 拼接提示词会触发 KeyError，导致待确认项直接变成空列表。后来改成 `.replace("{report}", report)`，只替换明确的占位符，问题消失。这类问题不复杂，但很能说明为什么结构化链路要逐段测试。

4.5 提示词的分工

提示词按场景拆开，不追求一个「万能提示词」。

场景	关键约束	稳定性措施
对话诊断	三段式结构，禁止输出实现步骤，先判断信息是否足够	系统提示词写死安全边界和工具使用规则
问卷出题	一次只问一个点，口语化，输出固定 JSON	去代码块加正则兜底，失败回落默认问题
报告生成	只使用问卷和对话中出现的信息，固定五段结构	冲突写入待确认，不替用户做假设
待确认项	输出 JSON 数组，区分模糊、缺失、冲突	逐项校验字段，非法项丢弃
评分	六维 0 到 100，风险最多 5 条	数值裁剪，等级回落
交付物与 A/B	只输出 JSON，字段为 Markdown 字符串	解析失败返回空结构，前端展示空状态

五、检索增强

5.1 知识库的来源

知识库是 12 篇原创 Markdown 文档，放在 `backend/kb`，主题覆盖需求文档、技术选型、开发流程、需求澄清、项目诊断、商业模式、角色权限、报价、工期、第三方服务与资质、常见风险和非功能需求。每篇控制在可读长度，标题就是主题标签，正文用短段落和清单写成，方便切片和检索。

这些文档不是模型生成的百科，而是我们按「需求方最常问什么」整理的检查清单。比如报价那篇会解释人力、设计、服务器、资质各自占什么位置；资质那篇会列出支付、备案、短信、地图等常见第三方服务。它们不保证覆盖所有行业，但能覆盖大部分中小型项目的第一轮判断。

5.2 切片与索引

运行时默认使用 `retrieval.py` 的本地 BM25。它按段落聚合，目标长度约 320 字，相邻片段保留 60 字重叠，最终把 12 篇文档切成 32 段。中文没有空格，直接用单字加二元切分代替分词器，不依赖 jieba 之类的额外库。

为了让小语料下的排序更稳，我们做了三件事：过滤「一个」「需要」「可以」这类高频虚词；标题命中时给 20% 的加权；同一篇文档最多取 2 段，避免 Top-3 全被同一主题占满。索引在首次检索时构建，之后常驻内存，12 篇文档的构建成本可以忽略。

如果配置了 `EMBEDDING_MODEL`，`build_kb.py` 会用 LangChain 的 RecursiveCharacterTextSplitter 按 400 字、80 字重叠切片，再写入 Chroma。向量库不是启动必需项，构建失败也只提示，不阻断应用。

5.3 两条检索路径

检索对外只有一个 `search` 入口，内部按当前条件选择路径。配置了 embedding 且向量库可用时走 Chroma 相似度检索；未配置、向量库缺失或调用异常时走 BM25。两条路径返回同样的片段结构，提示词组装层不需要知道底层用的是哪一种。

这次交付环境的 `/api/health` 返回 `mode: keyword`，说明当前没有启用向量库，所有检索都由 BM25 完成。这不是降级失败，而是默认配置下的正常状态。我们特意保留这个默认值，因为评审机器不一定能访问 embedding 网关，而本地检索必须保证可用。

5.4 降级不是口号

很多项目会把「降级」写在架构图里，但真正的降级需要满足三个条件：失败能被捕获、降级后的结果可用、用户能知道当前状态。

第一点靠异常捕获和懒加载。向量库只在首次使用时初始化，初始化失败不会影响应用启动。第二点靠统一的返回结构，BM25 和 Chroma 都返回 `source` 与 `content`，提示词不用改。第三点靠 `/api/health` 暴露 `mode`，排查时一眼能看出当前走的是哪条路径。

比赛现场最怕的不是效果差一点，而是演示到一半卡住。我们宁愿接受 BM25 召回率低一些，也不愿让整个诊断流程依赖一个可能超时的外部服务。

5.5 检索的局限

BM25 的优势是零依赖、可解释、速度快，劣势是只能匹配字面，无法理解同义表达。用户说「预算」能命中报价文档，说「要花多少钱」也能命中，但说「投入产出」时可能召回不到最相关的段落。知识库只有 12 篇，覆盖范围也有限，遇到冷门行业时回答会比较泛。

我们没有在比赛版本里强行调更大的模型或更多的知识库，而是把边界写清楚：检索结果只是提示词的参考，最终判断仍由模型完成；如果知识库没有覆盖，系统应该承认不知道，而不是编一个看似专业的答案。

六、核心功能实现

6.1 账号与鉴权

注册要求 11 位手机号，密码至少 8 位，并且包含大写字母、小写字母、数字、特殊字符中的至少三种。密码用 PBKDF2-SHA256 哈希，10 万次迭代，每个账号一个 16 字节随机盐，数据库里不保存明文。

登录成功签发 64 位十六进制 Token，服务端只保留每个账号最新的一条，旧登录自动失效。前端把 Token 存在 `localStorage`，所有请求统一加 `Authorization: Bearer`。接口返回 401 时，全局处理器清理会话并跳回登录页。

注销账号会级联删除该账号的 Token 和验收记录，但浏览器里的报告草稿需要前端同时清理。我们在个人中心做了提示，避免用户以为删号后本地数据也会自动消失。

6.2 问卷

问卷以一个种子问题开场，每次提交后由模型根据已答内容生成下一题。题型支持单选、多选和文本，用户可以回退上一题，也可以在题目下补充说明。信息足够时，模型主动收尾，给出小结并邀请生成报告。

这里最容易做错的是「一次问太多」。如果模型一上来就列出十个问题，用户会失去耐心。提示词里明确要求一次只问一个点，并且允许口语化表达。前端把问题卡做成逐步展开的形式，答完一题才出现下一题，减少压迫感。

问卷答案会同时进入 `SESSION_QA`，生成报告时作为附加上下文。用户如果中途切换到自由对话，之前答过的内容也会被拼进对话上下文，避免重复提问。

6.3 自由对话

自由对话直接进入 Agent。用户消息、知识库检索、模型回答以 NDJSON 事件流式传输，前端根据事件类型更新气泡、检索状态和思考提示。生成过程中可以随时停止，已经输出的内容会保留，并标注「已停止生成」。

Markdown 回答支持代码高亮、一键复制和外链新窗口打开。代码块由自定义 `fence` 渲染器输出「语言栏、复制按钮、代码」三部分，避免 markdown-it 默认渲染产生嵌套 `<pre>`，也避免复制按钮被包进代码区域。

对话的边界由系统提示词约束：不输出实现步骤，不替用户做技术选型，不承诺报价，遇到超出范围的问题直接说明。这些约束不是为了让回答变短，而是为了让产品始终停留在「需求诊断」的定位上。

6.4 报告与待确认事项

报告生成时会把问卷答案和自由对话合并，按「项目概述、已具备、还缺失、待确认问题清单、建议下一步」五段输出。提示词要求只使用用户提供的信息，遇到冲突或缺失就写进待确认，不替用户假设。

报告生成后，系统同步抽取待确认事项，逐条标注模糊、缺失或冲突。前端用卡片展示，支持「已确认」「不涉及」「补充说明」，并提供一键全部确认、已确认计数和进度条。全部决定后才能提交验收，避免漏项。

这一步是整个产品最像「顾问」的地方。模型不只是总结，而是把「你没说清楚的地方」单独拎出来，让用户逐条拍板。拍板结果会随验收记录一起保存，后续改需求时可以追溯当时确认了什么。

6.5 评分、交付物与 A/B 方案

六维评分从形态、用户、功能、预算、时间、合规六个维度给出 0 到 100 分，并附最多 5 条风险提示。评分不是目的，目的是让用户一眼看到短板。比如预算维度分数低，通常意味着用户只说了一个总数，没有拆清设计、开发、服务器和资质各占多少。

交付物模块一次生成 PRD、RFP 和成本工期估算。PRD 面向开发和设计，RFP 面向询价，成本工期面向决策。A/B 方案则给出 MVP 与完整版的对比，说明各自包含什么、适合什么阶段、推荐哪一个以及理由。

这些内容都由模型生成，但提示词要求它们必须引用报告中的信息，不能凭空补充功能。生成失败时返回空结构，前端展示「暂未生成」而不是一段看起来像真的假数据。

6.6 验收与版本

每次验收会计算报告正文和决策项的 SHA-256 指纹，按「账号加项目标识」递增版本号，并与上一版做行级 diff。凭证包含验收编号、账号、时间、版本、指纹和逐项结论，可以导出 Markdown。

版本号的作用不是好看，而是让「第几版确认了什么」有据可查。用户改完需求再次验收时，系统会返回与上一版的差异，哪些段落新增、哪些删除一目了然。指纹则用于判断两份报告是否完全一致，避免内容相同但版本号不同造成的混淆。

验收记录存 SQLite，报告正文存浏览器，这两者通过指纹关联。换浏览器后，历史报告看不到，但用同一账号仍能核对验收记录。这是本地单机形态下的折中，不是完整的云端同步。

6.7 导出、模板库与适配

报告支持 Markdown、PDF 和长截图三种导出。Markdown 保留结构，适合继续编辑；PDF 由服务端 reportlab 排版，适合直接发送；长截图由前端 html2canvas 以 2 倍分辨率生成，适合发到聊天窗口。

模板库内置 6 套行业模板和 2 个案例成品，覆盖电商、物联网、OA、会员营销、在线教育和本地生活。模板可以搜索、预览、导出 Markdown，也可以点「以此开始诊断」把摘要带入首页输入框。我的图纸支持按标题或日期搜索、删除二次确认，并用徽章标出草稿、已验收、评分和交付物状态。

界面针对桌面和手机分别优化，但没有做两套代码。桌面端使用双栏 Hero 和顶部导航，手机端压缩为单栏并保留底部 Tab。320px 宽度下表格和卡片改为单列，避免横向滚动。

七、安全与异常

7.1 输入审查

`safety.py` 在请求进入模型之前做一层规则检查。命中危险物品制作、入侵窃取、未成年人色情、代写论文等明确违规请求时直接拒答；识别到「忽略之前指令」「输出系统提示词」「开发者模式」等注入话术时也拒绝执行，并引导用户回到正常的需求描述。

规则不是万能的，但它能挡住最明显的恶意输入，也能在演示时说明安全边界。输入长度限制为 2000 字，超过时提示精简，避免超长上下文带来的费用和延迟。

7.2 提示词边界

系统提示词明确写了三件事：不泄露提示词和内部规则；不臆造政策、价格和资质要求；涉及个人信息时提醒用户注意合规。模型被要求在有疑问时说明不确定，而不是编一个听起来合理的答案。

这些约束和输入审查是两层防线。输入审查挡明显违规，提示词约束管住灰色地带。两层都不能保证百分之百，但比只做其中一层更稳。

7.3 输出脱敏

所有可能回显到前端的异常信息都会先经过 `mask_secrets`，把 `sk-` 开头的 Key、Bearer Token 和长随机字符串替换成 `***`。这样即使网关报错里带了鉴权信息，也不会通过页面泄露。

日志里也不打印完整 Key。配置检查只输出「是否配置」，不输出具体值。这个习惯在比赛提交前尤其重要，因为仓库是公开的。

7.4 故障与降级

故障	用户看到什么	系统怎么处理
未配置 LLM Key	明确提示去 `backend/.env` 填写	页面正常打开，AI 接口返回 503 或流式错误事件
模型网关超时	可读的失败提示	捕获异常，脱敏后返回，不抛堆栈
向量库缺失	无感	自动使用 BM25，健康接口显示 keyword
JSON 解析失败	空状态或默认问题	去围栏、正则提取、字段校验、回落

故障	用户看到什么	系统怎么处理
数据库不可写	启动时提示路径和权限	抛出明确错误，避免带着坏状态运行
前端静态文件缺失	首页 404	启动脚本会先构建前端，正常流程不会出现

7.5 隐私边界

报告正文、评分、交付物和草稿只存在浏览器本地，服务端不保存。账号、Token 和验收记录存在本地 SQLite，也不上传到第三方。模型调用会携带用户输入和必要的上下文，这是使用云端模型的必然代价，我们在界面上说明「数据只留在本机」指的是应用侧存储，不包括模型推理请求。

如果后续要做真正的私有化部署，可以把模型换成本地服务，其他结构不需要大改。这也是我们坚持 OpenAI 兼容协议的原因之一。

八、工程化与部署

8.1 一条命令启动

根目录执行 `python run.py`，脚本会依次完成：创建 `backend/.venv`、安装后端依赖、安装前端依赖、构建 `frontend/dist`、从 8000 起寻找可用端口、启动 FastAPI、打开浏览器。源码没改时会跳过前端构建，第二次启动通常只要几秒。

脚本处理了 Windows 的 `npm.cmd`、`taskkill` 和 UTF-8 输出，也保留了 macOS 和 Linux 的分支。评审老师不需要懂 Python 虚拟环境，只要能运行 Python 即可。

常用参数保留得不多：`--dev` 同时启动 Vite 和 FastAPI，`--port` 指定起始端口，`--rebuild` 强制重建前端，`--no-browser` 不自动打开浏览器。参数少是有意的，避免把启动器做成另一个需要学习的工具。

8.2 环境变量

变量	必填	默认值	说明
`LLM_API_KEY`	是	无	模型网关 Key，写在 `backend/.env` 或 `local_config.py`
`LLM_BASE_URL`	否	`https://maas.qianwenaiapi.com/compatible-mode/v1`	OpenAI 兼容网关
`LLM_MODEL`	否	`deepseek-v4.1-flash-1017`	模型型号
`LLM_TEMPERATURE`	否	`0`	采样温度，结构化输出建议保持 0
`LLM_ENABLE_THINKING`	否	不发送	设置 0 或 1 才显式传给网关
`EMBEDDING_MODEL`	否	空	配置后才启用向量检索，否则用 BM25
`EMBEDDING_API_KEY`	否	复用 LLM Key	向量模型独立网关时填写
`EMBEDDING_BASE_URL`	否	复用 LLM 地址	向量模型独立网关时填写
`DB_PATH`	否	`backend/data/jigou.db`	SQLite 文件位置，相对路径按 backend 解析
`MAX_INPUT_CHARS`	否	`2000`	单条输入上限
`PORT`	否	`8000`	仅直接运行

变量	必填	默认值	说明
			backend/app.py 时生效
VITE_API_TARGET	否	http://127.0.0.1:8000	前端开发代理目标

模板放在 backend/.env.example 和 frontend/.env.example。真实 Key 通过 .gitignore 排除，.env、local_config.py、数据库和向量库都不会进入仓库。

8.3 依赖与版本

后端依赖固定在 backend/requirements.txt，包括 FastAPI 0.141.1、Uvicorn 0.52.4、LangChain 1.4.0、LangGraph 1.2.11、langchain-openai 1.6.2、openai 3.8.0、Chroma 1.5.9 和 reportlab 5.0.1。前端依赖在 frontend/package.json 和 package-lock.json，Node 版本要求 20.19 以上。

固定的版本号不是为了好看，而是为了复现。LangChain 1.x 和 0.x 的接口差异很大，Chroma 的持久化格式也变过。比赛提交后如果评审环境重新安装依赖，锁版本能减少「在我机器上能跑」的概率。

8.4 构建与静态托管

前端使用 Vite 8 构建，vite-plugin-singlefile 把 JS 和 CSS 内联进一个 index.html，产物约 766 KB。这样 FastAPI 只需要返回一个文件，不需要处理静态资源路径。代价是首屏加载的 HTML 变大，但本地访问没有网络延迟，实际体验可以接受。

生产模式不启动 Vite。修改前端后必须重新 npm run build 或 python run.py --rebuild，否则 FastAPI 返回的还是旧产物。这个约束写在 README 和 AGENTS.md 里，避免后续维护者改了源码却看不到变化。

九、测试与质量

9.1 自动化检查

项目提交了 12 个基于 unittest 的接口与核心模块测试，运行命令是 `python -m unittest discover -s tests -v`，不依赖 pytest。测试库写在系统临时目录，不会污染 `backend/data/jigou.db`。

静态检查使用 `python -m compileall backend run.py` 和 `ruff check backend run.py`。前端用 `npm run build` 验证模板、导入和打包。当前代码的检查结果、测试用例和真实模型链路记录在《测试文档》中，这里不重复展开。

9.2 接口回归

接口测试覆盖健康检查、注册校验、重复注册、登录、Token 鉴权、登出、改密、注销、验收版本递增、账号隔离、PDF 导出、缺少 Key 的提示、安全规则和 BM25 检索。测试过程中确实发现过问题，例如早期用例共用同一个手机号，导致改密后其他用例登录失败；后来改成每个用例独立账号，测试才稳定。

这类问题看起来是测试代码的问题，但它提醒我们：账号状态是共享的，测试之间不能假设彼此独立。产品里的「一个账号只保留最新 Token」也是同样的逻辑，旧 Token 失效是设计，不是缺陷。

9.3 真实模型链路

除自动化用例外，我们还用真实网关跑了一次完整链路：输入奶茶店扫码点单小程序的三条问卷信息，问卷出题耗时约 10.6 秒，报告生成耗时约 58.8 秒，生成正文 852 字，抽取 9 条待确认事项，六维评分 62 分。另外用一段短报告验证验收和导出：首次验收得到 v1，修改后再验收得到 v2，diff 长度 230 字符；PDF 导出返回 3006 字节，文件头为 `%PDF`。

这次实测说明链路是通的，也暴露了一个现实问题：报告生成接近一分钟，用户需要明确的等待反馈。前端因此保留了流式对话和生成状态提示，问卷和报告按钮也有 loading 状态。

9.4 浏览器检查

浏览器端复核了登录、首页、模板库、我的图纸、个人中心、模板带入诊断和未登录跳转。登录后各页面没有控制台报错，当前视口下没有横向溢出；清空本地会话后访问 `#/history` 会被重定向到 `#/login?redirect=/history`，说明路由守卫生效。

更细的四档宽度检查和历史手工记录写在《测试文档》里。我们没有把开发阶段的所有截图都放进仓库，避免文档体积失控。

9.5 已知问题

当前没有发现阻断主流程的缺陷，但有几个需要说明的小问题。个人中心的注册时间依赖异步请求，页面刚打开时会短暂显示「—」，请求完成后才出现具体时间；这不是数据丢失，但视觉上不够稳。BM25 对同义表达不敏感，冷门行业可能召回不足。报告生成时间受网关影响，慢的时候接近一分钟。

这些问题不影响比赛演示，但会进入后续计划。

十、局限与后续

10.1 当前局限

最明显的局限是数据不跨设备。报告和草稿存在浏览器，换一台电脑就看不到历史；SQLite 只有账号、Token 和验收记录，无法单独恢复报告正文。这个设计换来了隐私和零运维，但牺牲了同步能力。

第二个局限是检索质量。默认 BM25 只能做字面匹配，知识库只有 12 篇，覆盖面有限。配置 embedding 后效果会好一些，但当前交付环境没有启用向量库，所以实际表现以关键词检索为准。

第三个局限是工程成熟度。前端使用 JavaScript，没有 TypeScript 的类型约束；自动化测试集中在后端，前端仍以手工检查为主；没有 CI，也没有容器化部署。对于比赛作品这些可以接受，对于长期维护还需要补。

10.2 下一步

如果继续做，我们优先补三件事。第一，把报告正文迁移到 SQLite 或提供导出导入，解决换设备丢失的问题。第二，补前端自动化测试，至少覆盖登录、问卷、报告和模板带入这几条主路径。第三，接入视觉模型，让用户可以直接上传截图或竞品页面做诊断，这比纯文本更接近真实需求场景。

更长期的方向是把诊断结果接到真实的服务商生态，例如根据地区、预算和行业推荐可合作的开发团队。但这涉及资质审核和商业合作，不适合在当前版本里仓促加入。

附录 A 目录结构

```
JGchuanzhibei/
├── run.py                    一键启动：依赖、构建、服务、浏览器
├── backend/
│   ├── app.py                FastAPI 入口、鉴权与业务接口
│   ├── agent_core.py         提示词、Agent、结构化生成、事件协议
│   ├── retrieval.py          BM25 关键词检索（向量不可用时兜底）
│   ├── safety.py             输入审查、注入拦截、密钥脱敏
│   ├── config.py             环境变量与路径配置
│   ├── db.py                 SQLite：账号、Token、验收记录
│   ├── pdf_export.py         Markdown 转 PDF
│   ├── build_kb.py           可选：构建向量库
│   ├── seed_demo.py          创建评审测试账号
│   ├── kb/                   12 篇知识库
│   └── requirements.txt
├── frontend/
│   ├── src/views/            登录、注册、对话、报告、模板、历史、个人中心
│   ├── src/components/       对话气泡、问题卡、评分卡、文档卡等
│   ├── src/stores/           账号、对话、问卷、归档、草稿
│   ├── src/api/              接口封装与 NDJSON 解析
│   └── vite.config.js
├── docs/
│   ├── md/                   Markdown 源文件
│   ├── word/                 Word 版
│   ├── pdf/                  PDF 版
│   └── images/               作品介绍配图
└── demo_video/               演示视频、静态页总览 PPTX
```

附录 B 接口清单

接口	方法	鉴权	说明
`/api/register`	POST	否	注册，手机号加强密码
`/api/login`	POST	否	登录并签发 Token
`/api/logout`	POST	是	使当前 Token 失效
`/api/me`	GET	是	当前账号信息
`/api/change_password`	POST	是	修改密码
`/api/delete_account`	POST	是	注销并级联删除数据
`/api/chat`	POST	是	流式对话，NDJSON
`/api/questionnaire`	POST	是	生成下一题
`/api/generate_doc`	POST	是	生成诊断报告、待确认项和评分
`/api/accept`	POST	是	保存验收记录与版本
`/api/assessment`	POST	是	六维评分与风险
`/api/deliverables`	POST	是	PRD、RFP、成本工期
`/api/ab_plan`	POST	是	A/B 方案
`/api/revision`	POST	是	按意见修订报告
`/api/export_pdf`	POST	是	导出 PDF
`/api/health`	GET	否	模型、知识库、数据库自检

附录 C 团队信息

项目	内容
团队名称	极构团队
所在学校	沧州交通学院
参赛组别	B 组
团队成员	贾宇航、吴思彤、马翌焜
指导教师	王玉潇
作品名称	极构 · 需求诊断顾问
代码仓库	https://github.com/ynamlu/JiGou